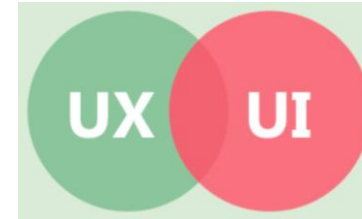




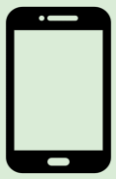
UX/UI for Mobile : Ergonomic Principles, Material You & Accessibility



Alix Tuloup & Victoire Bernard de
Dompsure

19/10/2025

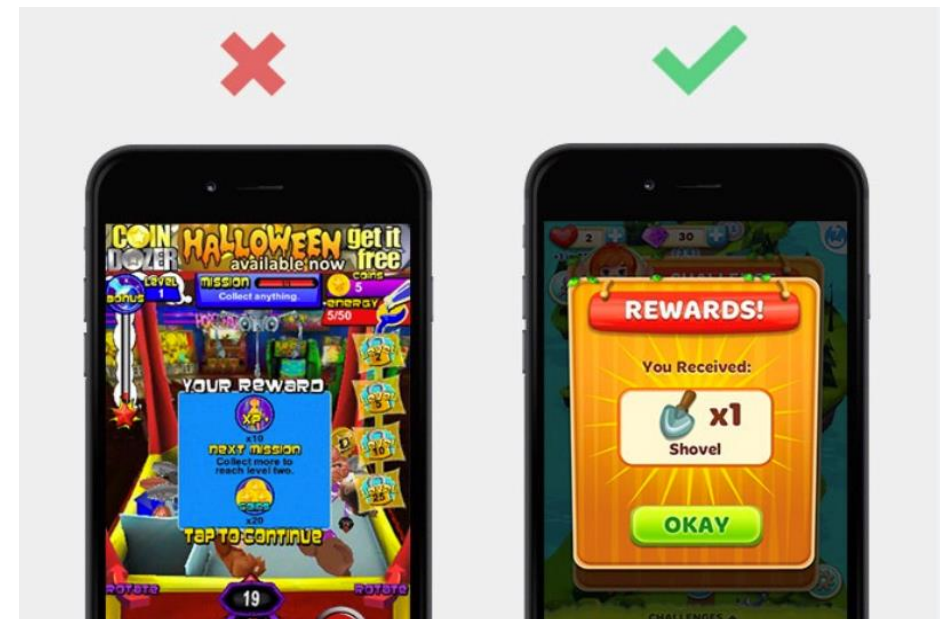
II.3510 – Mobile Application
Development for Android

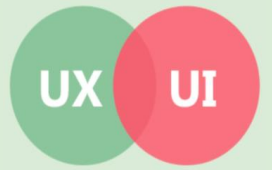
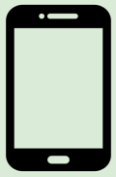


Why UX/UI Matters in Mobile Development

What frustrates you most when using a mobile app?

- Mobile apps are used everywhere, every time
- 90% of app uninstalls are due to bad UX or unclear UI
- Small screens and touch navigation = new ergonomic challenges
- Good UX -> more engagement, loyalty, and positive reviews
- Poor UX -> frustration, bad ratings, and uninstalls





What is UX ? What is UI ?

UI

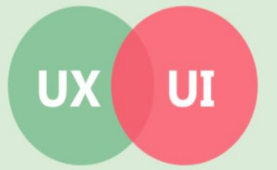
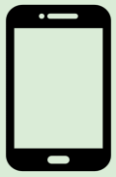
- “User Interface” = how the app looks
- Consistent color palette and typography
- Proper spacing and alignment for readability
- Interactive elements that look “clickable”
- Visual hierarchy (important things stand out)



- Great apps balance both, aesthetic and usability
- Example: a beautiful app (UI) can still be frustrating (bad UX)

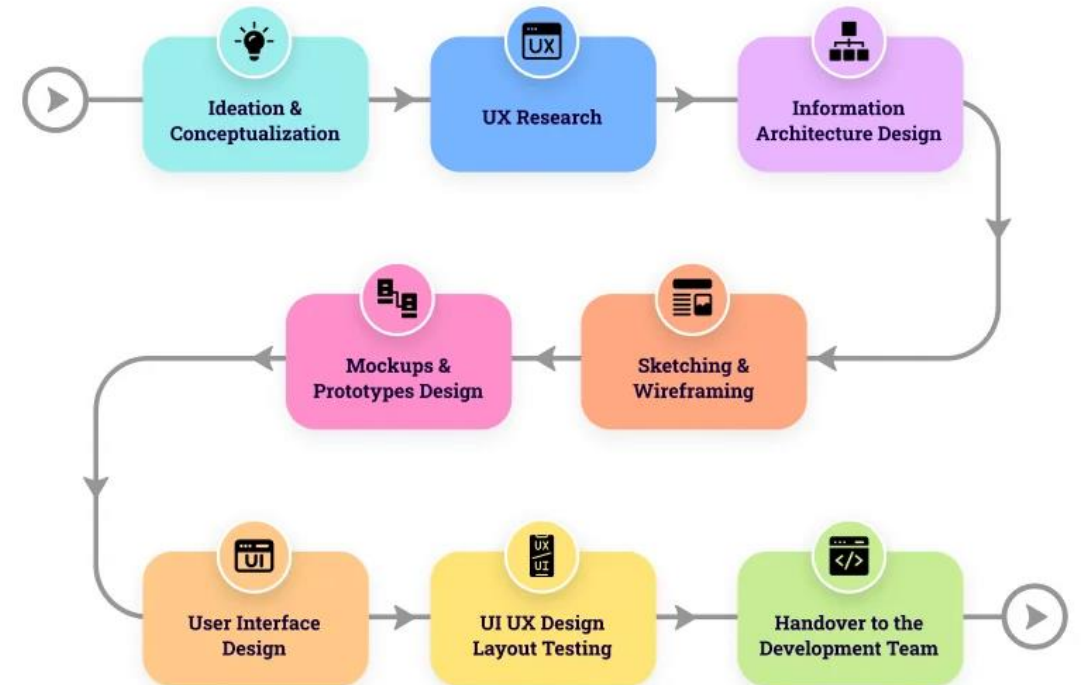
UX

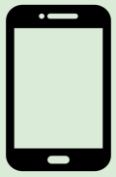
- “User Experience” = how the app feels to use
- Logical flow between screens
- Clear goals and feedback (knowing what’s happening)
- Emotional connection and comfort
- Accessibility and inclusivity for all users



UX VS UI : What's the difference ?

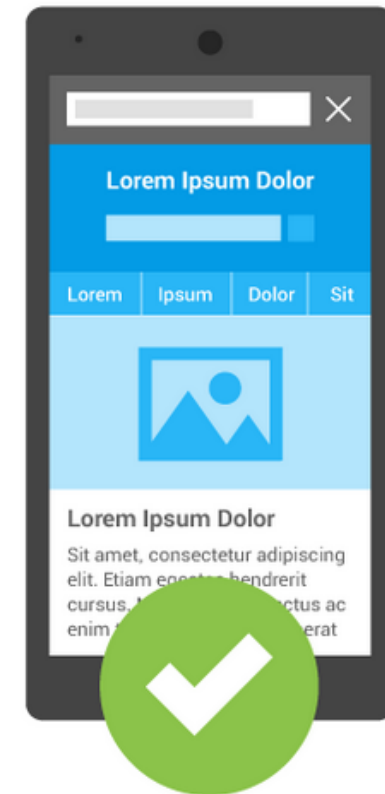
UX (Experience)	UI (Interface)
App structure and logic	Colors and shapes
User journey	Buttons and icons
Ease of navigation	Visual aesthetics
Feels intuitive	Feels beautiful

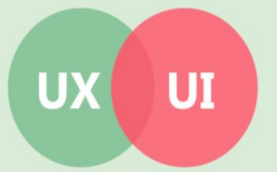
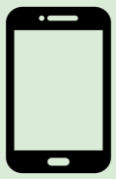




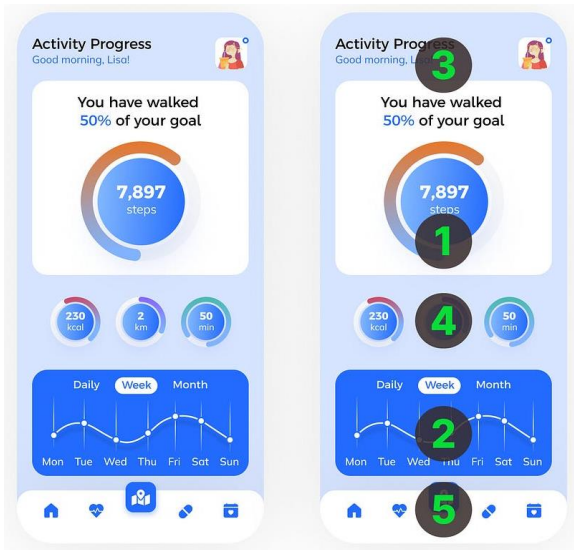
Mistakes to Avoid

- Small or too-close buttons
- Too many buttons / cluttered layout
- Poor contrast or small text
- Hidden navigation or confusing icons
- No accessibility attributes or feedback
- Ignoring tablet and foldable formats

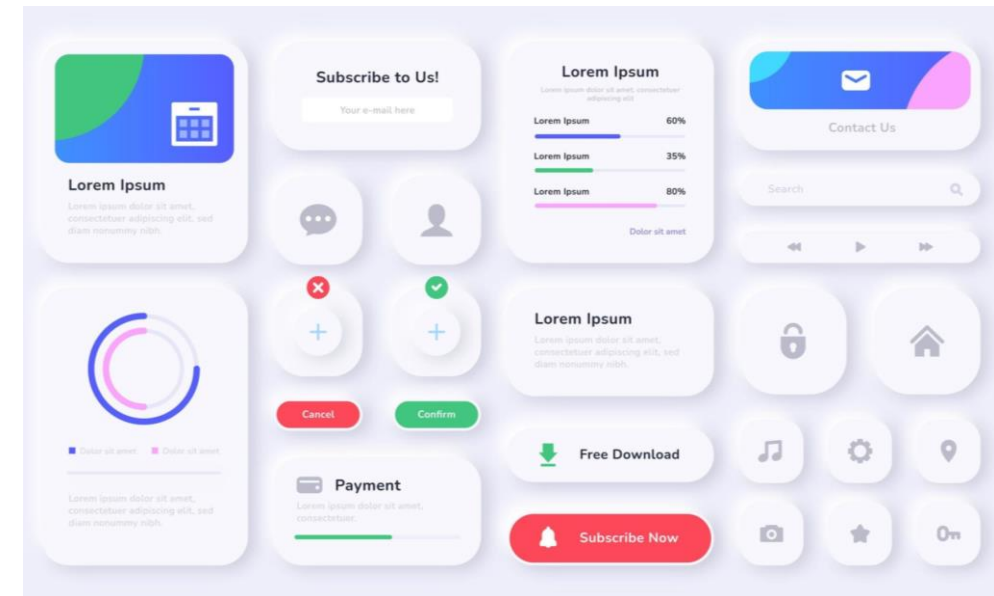


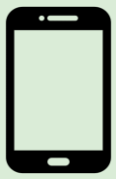


Design rules that guide good mobile UX/UI



- Simplicity and minimalism
- Consistency across screens
- Visual hierarchy (important elements first)
- Feedback on user actions (button animation)
- Accessibility for all users





Accessibility = Inclusive Design

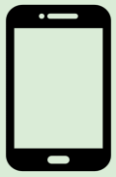
1 in 6 people lives with a disability (vision, motor, cognitive, hearing).

Accessibility improves **usability for everyone**, not just disabled users.

Accessible apps have **higher retention and fewer errors**.

Required by **international guidelines** (WCAG, ADA...).





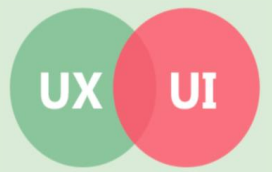
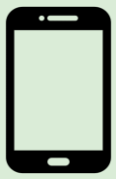
Design for everyone



- Design usable **by everyone**
- Support for **screen readers** and voice commands
- **High contrast & readable text**
- **Large touch areas** for easy interaction
- **Simple layouts & clear navigation**

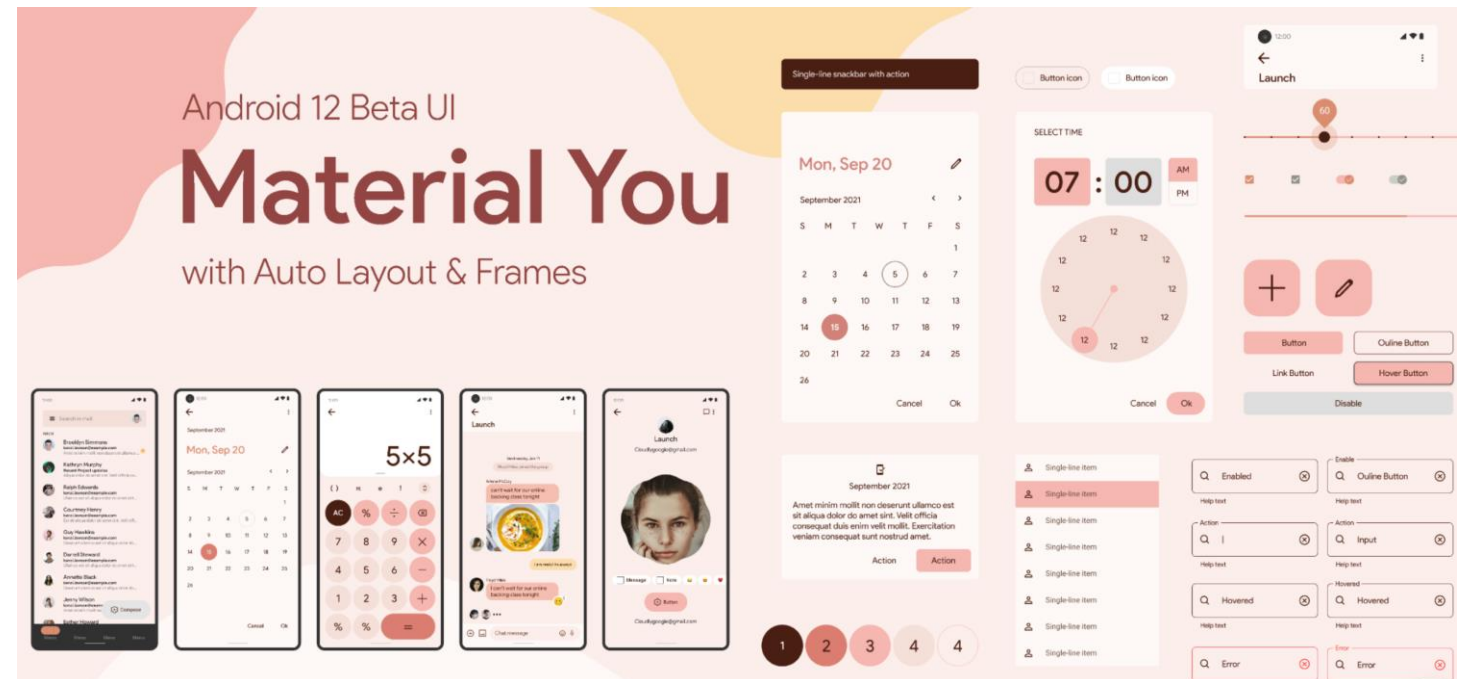
The Four Principles of Accessibility

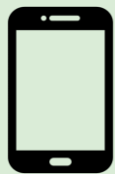
P	Perceivable
O	Operable
U	Understandable
R	Robust



Material You - Google's Modern Design System

- **Dynamic system** that adapts the UI to the user's device colors & preferences
- **Focus on personalization**, expressive design, and accessibility
- **Follows clear rules for spacing**, motion, harmony, and readability
- **Official design language** for Android 12+





Core Principles of Material You

Dynamic color

Rounded components that reflect user personality

Meaningful motion

UI colors adapt to the system wallpaper

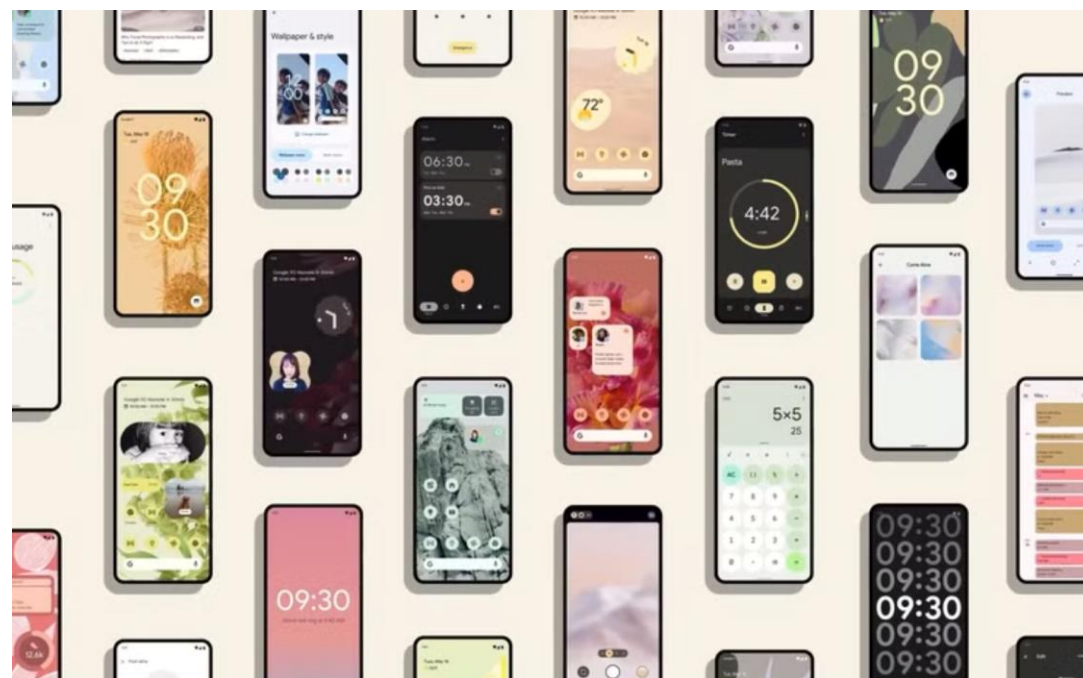
Consistent spacing & hierarchy

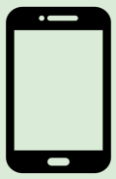
Smooth transitions to support comprehension

Shape flexibility

Clear structure that guides the eyes naturally

Contrast, readable font scaling, large touch targets





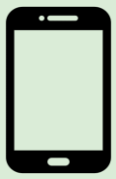
How to enable Material You in an Android app

- Automatically adapts colors to the system palette (Material You)
- Ensures proper contrast and accessibility
- Removes the need for manually defining large color sets

```
@Composable
fun MyTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit
) {
    val context = LocalContext.current

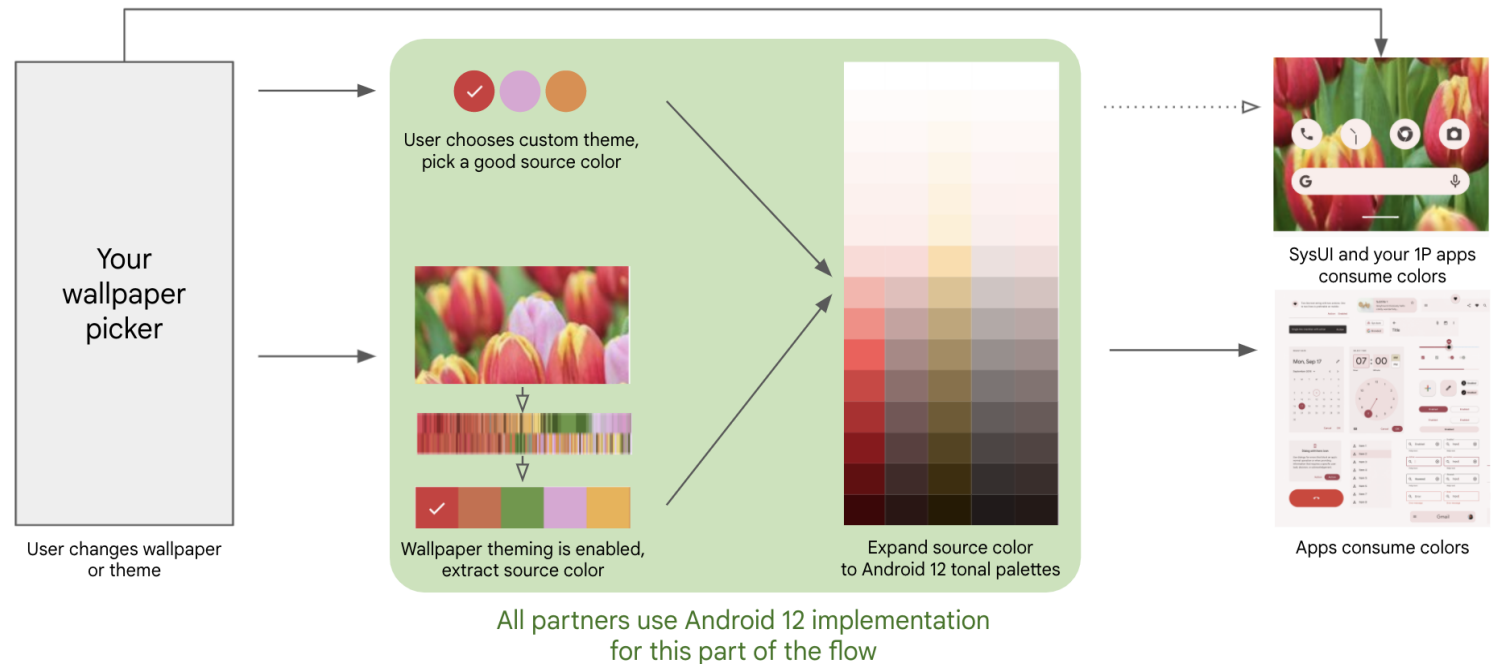
    val colorScheme =
        if (dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
            if (darkTheme) dynamicDarkColorScheme(context)
            else dynamicLightColorScheme(context)
        } else {
            if (darkTheme) darkColorScheme()
            else lightColorScheme()
        }

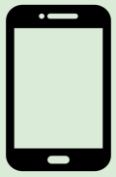
    MaterialTheme(
        colorScheme = colorScheme,
        typography = Typography(),
        content = content
    )
}
```



Material You components in practice

- Buttons adapt **shape + elevation + dynamic color**
- Navigation bars **follow system theme**
- Cards use **rounded corners & content hierarchy**
- Typography **automatically** scales with user settings
- Icons inherit colorScheme highlights





Démonstration

